

Сравнение оптимизаторов Vision transformer

1. Введение

Vision transformers стали незаменимой частью компьютерного зрения. Благодаря своей архитектуре патчи могут обмениваться информацией друг с другом, позволяя нейронной сети понимать как разные части изображения связаны между собой. Однако, важной частью обучения является не только архитектура сети, но и качество данных, loss функции и оптимизаторы весов. В данной работе сравниваются два оптимизатора: Lion и AdamW.

2. Описание метода

Сравнение проводилось на датасете Oxford-III Pet, который содержит 37 классов по 200 цветных изображений. Метрикой сравнения является batch loss – ошибка классификатора на батче из 4 картинок. Данная метрика выбрана из-за ограниченных компьютерных ресурсов и, следовательно, малым кол-вом эпох тренировки трансформера. В качестве оптимизатора весов сравниваются два алгоритма: Lion и AdamW. Оба алгоритма используют одинаковое значение learning rate: $1e-6$.

2.1 Lion

Lion – Evolved Sign Momentum отслеживает только градиентную скользящую среднюю (momentum), что позволяет снизить нагрузку на память. Алгоритм менее чувствителен к выбросам и более плавно обновляет веса. На вход алгоритм принимает следующие параметры:

- θ_0 - веса нейросети
- $f(\theta)$ – функция потерь, loss функция
- γ_t – скорость обучения, learning rate в момент t
- β_1, β_2 – коэффициенты накопления
- λ – коэффициент затухания весов, weight decay

Моментум принимает начальное значение ноль: $m_0 \leftarrow 0$. В каждый момент времени t ($t \in \mathbb{N}$) изменяются веса для минимизации функции потерь с учетом прошлых обновлений:

$$\begin{aligned} g_t &\leftarrow \nabla_{\theta} f(\theta_{t-1}) \\ u &\leftarrow \beta_1 m_{(t-1)} + (1 - \beta_1) g_t \\ m_t &\leftarrow \beta_2 m_{(t-1)} + (1 - \beta_2) g_t \\ \theta_t &\leftarrow \theta_{t-1} - \gamma_t (\text{sign}(u) + \lambda \theta_{t-1}) \end{aligned}$$

Высчитывается градиент loss функции $f(\theta_{t-1})$ по отношению к весам нейронной сети. В векторе u хранится информация о том, насколько сильно сохраняется информация с прошлых шагов оптимизации. Также, обновляется моментум с использованием второго коэффициента накопления β_2 . Затем, $\text{sign}(u)$ преобразует вектор в положительные и отрицательные единицы в зависимости от знака начальных значений, что позволяет изменять веса на одинаковую величину. Большие веса уменьшаются за счет коэффициента затухания весов γ_t и веса нейронной сети обновляются с учетом скорости обучения.

2.2 AdamW

AdamW(Adam with decoupled weight decay) – алгоритм, который происходит от Adam(Adaptive Moment Estimation) и вычисляет скорость обучения для каждого параметра, используя экспоненциально затухающее скользящее среднее градиентов и их квадратов. На вход принимаются те же параметры, что и в Lion, но добавляется параметр эпсилон ϵ . Первый и второй моментум принимают нулевые значения при инициализации.

В каждый момент времени t ($t \in \mathbb{N}$):

$$\begin{aligned} g_t &\leftarrow \nabla_{\theta} f_t(\theta_{(t-1)}) - \lambda \theta_{t-1} \\ m_t &\leftarrow \beta_1 m_{t-1} + (1 - \beta_1) g_t \\ v_t &\leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \\ \widehat{m}_t &\leftarrow m_t / (1 - \beta_1^t) \end{aligned}$$

$$\widehat{v}_t \leftarrow v_t / (1 - \beta_2^t)$$

$$\theta_t \leftarrow \theta_{(t-1)} - \gamma_t (\widehat{m}_t / (\sqrt{\widehat{v}_t} + \epsilon) + \lambda \theta_{t-1})$$

Так же считается градиент loss-функции $f_t(\theta_{(t-1)})$ по отношению к весам. Вектор m_t принимает значение инерции движения, а в векторе v_t сохраняется информация о изменчивости градиентов с использованием второго коэффициента накопления β_2 . Затем значения m_t и v_t корректируются, чтобы исправить смещение на первых шагах оптимизации. Деление среднего градиента на корень из его изменчивости позволяет адаптировать шаг под каждый параметр: там, где градиент скачет, шаг уменьшается. В финале веса нейронной сети обновляются с учетом скорости обучения γ_t , а коэффициент затухания λ уменьшает большие веса.

3. Гиперпараметры

Не указанные гиперпараметры принимают значения по умолчанию:

PatchEmbedding(in_channels=3, embed_dim=768, patch_size=16)
ViT(image_size=224, embed_dim=768, in_channels=3, patch_size=16, dropout=0.1, num_heads=12, ff_dim=3072, depth=12, num_classes=1000)
AdamW(learning_rate=1e-5)
Lion(learning_rate=1e-5)
Epochs = 10

4. Результаты

По результатам сравнения batch loss при использовании Lion и AdamW оптимальным алгоритмом при заданных гиперпараметрах оказался AdamW. На графике AdamW/batch loss прослеживается линейная зависимость batch loss от кол-во тренировки, в то время как Lion не снижает ошибку.

